

LAB ASSIGNMENT – 11.5

NAME – SHREE PRIYA

2303A51465

BATCH – 29

Task Description #1 (Refactoring – Removing Global Variables)

- Task: Use AI to eliminate unnecessary global variables from the code.

- Instructions:

- o Identify global variables used across functions.
- o Refactor the code to pass values using function parameters.
- o Improve modularity and testability.

- Sample Legacy Code:

```
rate = 0.1
```

```
def calculate_interest(amount):
```

```
    return amount * rate
```

```
print(calculate_interest(1000))
```

- Expected Output:

- o Refactored version passing rate as a parameter or using a configuration structure.

Refactor the given Python code to remove the global variable and pass the value as a function parameter to improve modularity and testability

```
def calculate_interest(amount, rate):
```

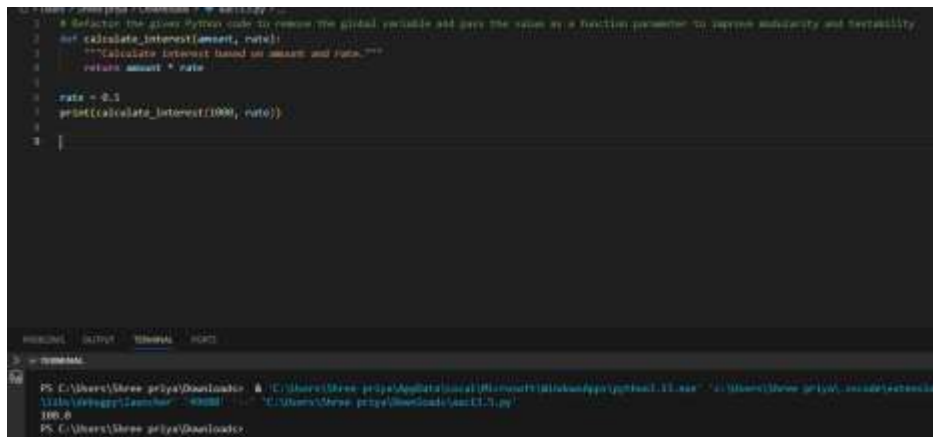
```
    """Calculate interest based on amount and rate."""
```

```
    return amount * rate
```

```
rate = 0.1
```

```
print(calculate_interest(1000, rate))
```

```
1 # Refactor the given Python code to remove the global variable and pass the value as a function parameter to improve modularity and readability.
2 def calculate_interest(amount, rate):
3     """Calculate interest based on amount and rate."""
4     return amount * rate
5
6 rate = 0.1
7 print(calculate_interest(1000, rate))
8
9 [
```



```
100.0
11 ]
```

Task Description #2 : (Refactoring Deeply Nested Conditionals)

- Task: Use AI to refactor deeply nested if–elif–else logic into a cleaner structure.

- Focus Areas:

- o Readability
- o Logical simplification
- o Maintainability

Legacy Code:

```
score = 78
```

```
if score >= 90:
```

```
    print("Excellent")
```

```
else:
```

```
    if score >= 75:
```

```
        print("Very Good")
```

```
    else:
```

```
        if score >= 60:
```

```
            print("Good")
```

```
        else:
```

```
            print("Needs Improvement")
```

Expected Outcome:

- o Flattened logic using guard clauses or a mapping-based approach.

Refactor the given Python code to remove deeply nested if–else statements and simplify the logic for better readability and maintainability.

score = 78

if score >= 90:

 print("Excellent")

elif score >= 75:

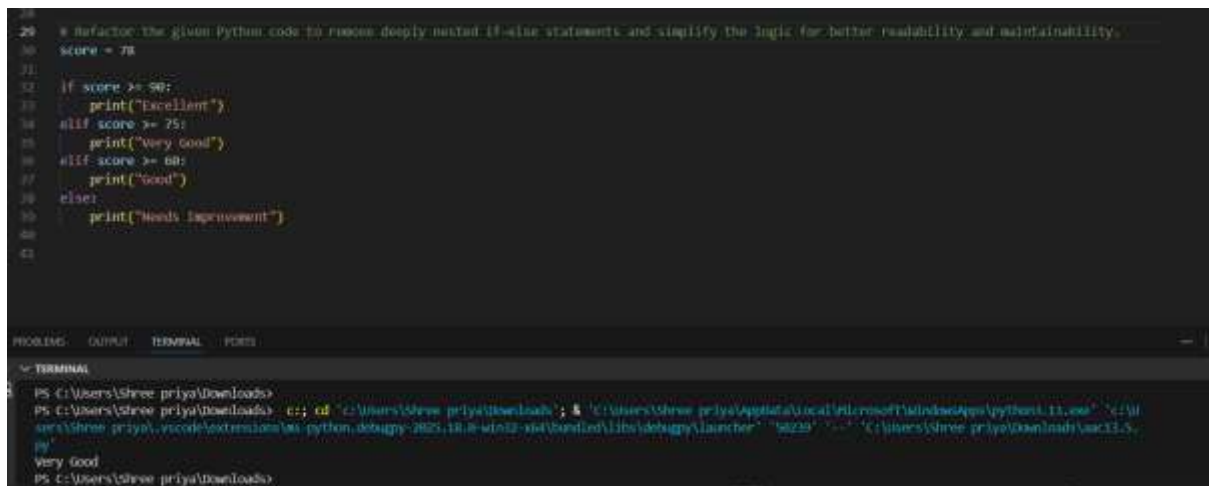
 print("Very Good")

elif score >= 60:

 print("Good")

else:

 print("Needs Improvement")



The screenshot shows a code editor with a dark theme. At the top, a comment reads: `# Refactor the given Python code to remove deeply nested if-else statements and simplify the logic for better readability and maintainability.` Below this, the Python code is written: `score = 78`, `if score >= 90:` `print("Excellent")`, `elif score >= 75:` `print("Very Good")`, `elif score >= 60:` `print("Good")`, and `else:` `print("Needs Improvement")`. Below the code editor is a terminal window. The terminal shows the command `cd "C:\Users\Shree priya\Downloads"` and the execution of the Python script using `python`. The output of the script is `Very Good`.

Task 3 (Refactoring Repeated File Handling Code)

- Task: Use AI to refactor repeated file open/read/close logic.

- Focus Areas:

- o DRY principle
- o Context managers
- o Function reuse

Legacy Code:

f = open("data1.txt")

print(f.read())

f.close()

f = open("data2.txt")

```
print(f.read())
```

```
f.close()
```

Expected Outcome:

o Reusable function using with open() and parameters

Refactor the repeated file open/read/close code using a reusable function and Python context manager (with open()) to follow the DRY principle

```
def read_file(filename):
```

```
    """Read and print contents of a file."""
```

```
    try:
```

```
        with open(filename, "r") as f:
```

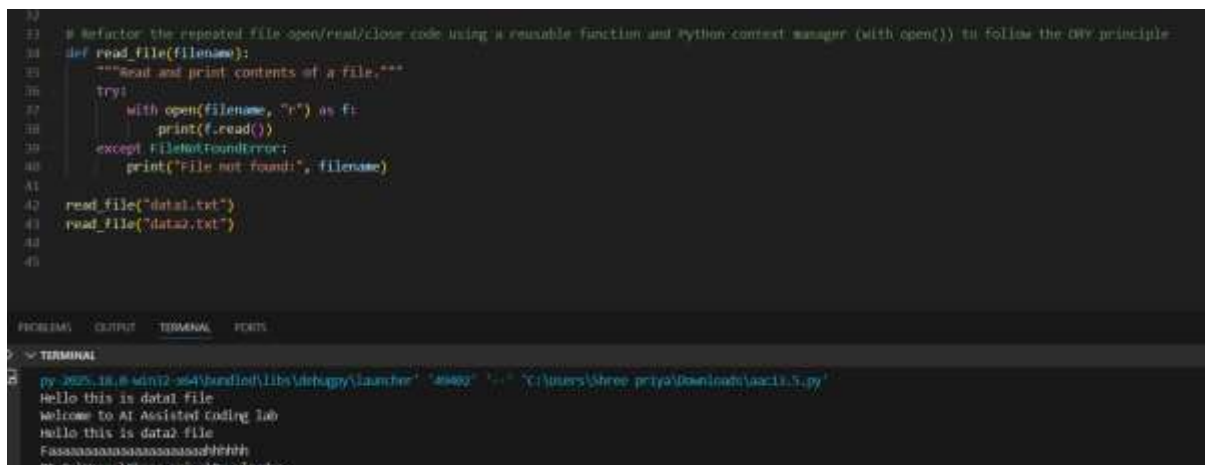
```
            print(f.read())
```

```
    except FileNotFoundError:
```

```
        print("File not found:", filename)
```

```
read_file("data1.txt")
```

```
read_file("data2.txt")
```



```
33 # Refactor the repeated file open/read/close code using a reusable function and Python context manager (with open()) to follow the DRY principle
34 def read_file(filename):
35     """Read and print contents of a file."""
36     try:
37         with open(filename, "r") as f:
38             print(f.read())
39     except FileNotFoundError:
40         print("File not found:", filename)
41
42 read_file("data1.txt")
43 read_file("data2.txt")
44
45
```

PROBLEMS OUTPUT TERMINAL FONTS

TERMINAL

```
py -3.10.8-win32-x64-handled\libs\debugpy\launcher -w -x C:\Users\Shree priya\Downloads\aac13.5.py
Hello this is data1 file
welcome to AI Assisted Coding lab
Hello this is data2 file
F
PS C:\Users\Shree priya\Downloads>
```

Task 4 (Optimizing Search Logic)

- Task: Refactor inefficient linear searches using appropriate data structures.
- Focus Areas:
 - o Time complexity
 - o Data structure choice

Legacy Code:

```
users = ["admin", "guest", "editor", "viewer"]

name = input("Enter username: ")

found = False

for u in users:
    if u == name:
        found = True

print("Access Granted" if found else "Access Denied")
```

Expected Outcome:

o Use of sets or dictionaries with complexity justification.

Refactor the given Python code to replace linear search with a set or dictionary for faster lookup and better time complexity.

```
users = {"admin", "guest", "editor", "viewer"}
```

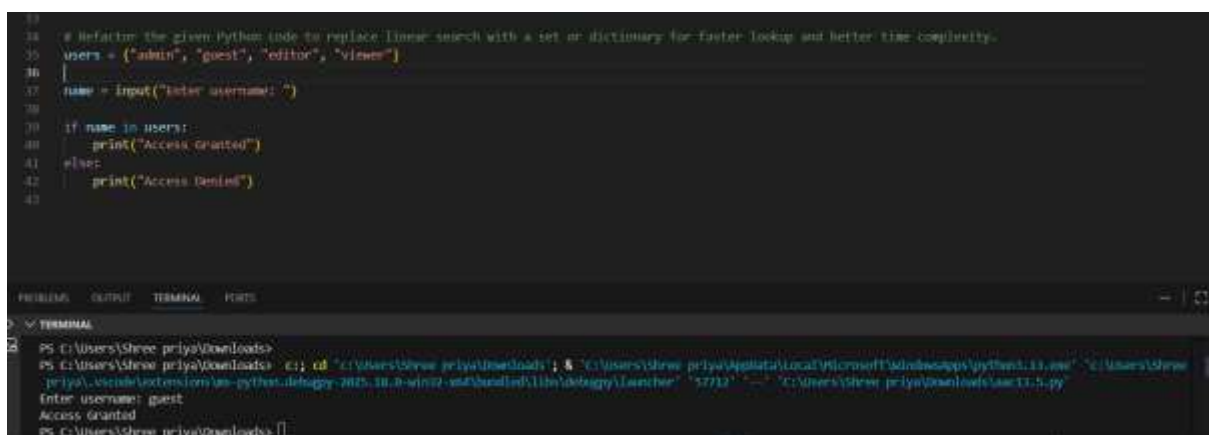
```
name = input("Enter username: ")
```

```
if name in users:
```

```
    print("Access Granted")
```

```
else:
```

```
    print("Access Denied")
```



The screenshot displays a code editor with the following Python code:

```
34 # Refactor the given Python code to replace linear search with a set or dictionary for faster lookup and better time complexity.
35 users = {"admin", "guest", "editor", "viewer"}
36
37 name = input("Enter username: ")
38
39 if name in users:
40     print("Access Granted")
41 else:
42     print("Access Denied")
43
```

Below the code editor, a terminal window shows the execution of the program:

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> cd "C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "C:\Users\Shree priya\Downloads\extension\ms-python.debugpy-2025.10.0-win32-and-bundled\lib\debugpy\launcher" "5712" -c "C:\Users\Shree priya\Downloads\src\11.5.py"
Enter username: guest
Access Granted
PS C:\Users\Shree priya\Downloads>
```

Task 5 (Refactoring Procedural Code into OOP Design)

- Task: Use AI to refactor procedural code into a class-based design.

- Focus Areas:

- o Object-Oriented principles

- o Encapsulation

Legacy Code:

```
salary = 50000
```

```
tax = salary * 0.2
```

```
net = salary - tax
```

```
print(net)
```

Expected Outcome:

- o A class like EmployeeSalaryCalculator with methods and attributes

Refactor the procedural Python code into an object-oriented class design using encapsulation and methods.

```
class EmployeeSalaryCalculator:
```

```
    """Class to calculate employee net salary after tax."""
```

```
    def __init__(self, salary):
```

```
        self.salary = salary
```

```
        self.tax_rate = 0.2
```

```
    def calculate_tax(self):
```

```
        """Calculate tax amount."""
```

```
        return self.salary * self.tax_rate
```

```
    def calculate_net_salary(self):
```

```
        """Calculate net salary after tax."""
```

```
        tax = self.calculate_tax()
```

```
        return self.salary - tax
```

```
emp = EmployeeSalaryCalculator(50000)
```

```
print(emp.calculate_net_salary())
```

```
34 # Refactor the procedural Python code into an object-oriented class design using encapsulation and methods.
35 class EmployeeSalaryCalculator:
36     """Class to calculate employee net salary after tax."""
37
38     def __init__(self, salary):
39         self.salary = salary
40         self.tax_rate = 0.2
41
42     def calculate_tax(self):
43         """Calculate tax amount."""
44         return self.salary * self.tax_rate
45
46     def calculate_net_salary(self):
47         """Calculate net salary after tax."""
48         tax = self.calculate_tax()
49         return self.salary - tax
50
51 emp = EmployeeSalaryCalculator(50000)
52 print(emp.calculate_net_salary())
53
54
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> .C
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> cd; cd 'C:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\Windows\apps\python3.11.exe' "C:\Users\Shree priya\code\extensions\vs-python-debugpy-2025.18.0-win32-x64\bundled\Libs\debugpy\launcher" "64343" -r "C:\Users\Shree priya\Downloads\sac13.5.py"
50000.0
PS C:\Users\Shree priya\Downloads> []
```

Task 6 (Refactoring for Performance Optimization)

- Task: Use AI to refactor a performance-heavy loop handling large data.

- Focus Areas:

- o Algorithmic optimization
- o Use of built-in functions

Legacy Code:

```
total = 0
```

```
for i in range(1, 1000000):
```

```
    if i % 2 == 0:
```

```
        total += i
```

```
print(total)
```

Expected Outcome:

- o Optimized logic using mathematical formulas or comprehensions, with time comparison.

Refactor the given Python loop to optimize performance using built-in functions or mathematical logic instead of iterating through all numbers.

```
total = sum(i for i in range(1, 1000000) if i % 2 == 0)
```

```
print(total)
```

```
53 # Refactor the given Python loop to optimize performance using built-in functions or mathematical logic instead of iterating through all numbers.
54 total = sum(i for i in range(1, 1000000) if i % 2 == 0)
55 print(total)
56
```

IDE LENS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads>
PS C:\Users\Shree priya\Downloads> cd 'c:\Users\Shree priya\Downloads'; & 'C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python2.7.exe' 'c:\Users\Shree priya\vscode\extensions\ms-python.debugpy-2025.12.0-win32-x64\bin\lib\debugpy\launcher' '54282' '-' 'C:\Users\Shree priya\Downloads\src11.3.py'
200000500000
PS C:\Users\Shree priya\Downloads>
```

Task 7 (Removing Hidden Side Effects)

- Task: Refactor code that modifies shared mutable state.

- Focus Areas:

- o Functional-style refactoring

- o Predictability

Legacy Code:

```
data = []
```

```
def add_item(x):
```

```
    data.append(x)
```

```
    add_item(10)
```

```
    add_item(20)
```

```
    print(data)
```

Expected Outcome:

- o Refactored function returning values instead of mutating globals

Refactor the code to remove hidden side effects by avoiding modification of global variables and returning values from the function

```
def add_item(data, x):
```

```
    """Return a new list with the added item."""
```

```
    return data + [x]
```

```
data = []
```

```
data = add_item(data, 10)
```

```
data = add_item(data, 20)
```


print(data)

```
53. # Refactor the code to remove hidden side effects by avoiding modification of global variables and returning values from the function
54. def add_item(data, x):
55.     """Return a new list with the added item."""
56.     return data + [x]
57.
58. data = []
59.
60. data = add_item(data, 10)
61. data = add_item(data, 20)
62.
63. print(data)
64. |
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> & "C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.11.exe" "C:\Users\Shree priya\.vscode\extensions\ms-python.debug
py-2025.10.9-ad112-ant\binaries\linux\debugpy\launcher" "56758" "--" "C:\Users\Shree priya\Downloads\acc11.5.py"
[10, 20]
PS C:\Users\Shree priya\Downloads>
```

Task 8 (Refactoring Complex Input Validation Logic)

- Task: Use AI to simplify and modularize complex validation

rules.

- Focus Areas:

- o Readability

- o Testability

Legacy Code:

```
password = input("Enter password: ")
```

```
if len(password) >= 8:
```

```
if any(c.isdigit() for c in password):
```

```
if any(c.isupper() for c in password):
```

```
print("Valid Password")
```

```
else:
```

```
print("Must contain uppercase")
```

```
else:
```

```
print("Must contain digit")
```

```
else:
```

```
print("Password too short")
```

Expected Outcome:

- o Separate validation functions with clear responsibility

Refactor the password validation code by separating validation rules into functions to improve readability and testability.

```
def is_length_valid(password):
```

```
    return len(password) >= 8
```

```
def has_digit(password):
```

```
    return any(c.isdigit() for c in password)
```

```
def has_uppercase(password):
```

```
    return any(c.isupper() for c in password)
```

```
password = input("Enter password: ")
```

```
if not is_length_valid(password):
```

```
    print("Password too short")
```

```
elif not has_digit(password):
```

```
    print("Must contain digit")
```

```
elif not has_uppercase(password):
```

```
    print("Must contain uppercase")
```

```
else:
```

```
    print("Valid Password")
```

```
52 # Refactor the password validation code by separating validation rules into functions to improve readability and testability.
53
54 def is_length_valid(password):
55     return len(password) >= 8
56
57 def has_digit(password):
58     return any(c.isdigit() for c in password)
59
60 def has_uppercase(password):
61     return any(c.isupper() for c in password)
62
63
64 password = input("Enter password: ")
65
66 if not is_length_valid(password):
67     print("Password too short")
68 elif not has_digit(password):
69     print("Must contain digit")
70 elif not has_uppercase(password):
71     print("Must contain uppercase")
72 else:
73     print("Valid Password")
74
```

PROBLEMS OUTPUT TERMINAL PORTS

TERMINAL

```
PS C:\Users\Shree priya\Downloads> cd "C:\Users\Shree priya\Downloads" & & "C:\Users\Shree priya\AppData\Local\Microsoft\WindowsApps\python3.12.exe" "C:\Users\Shree priya\Downloads\python_debuggy-2025.10.0-win32-x64\bundle\libs\debuggy\launcher" "52468" "C:\Users\Shree priya\Downloads\src\3.py"
Enter password: abc12345
Must contain uppercase
```

JUSTIFICATIONS:

Task 1 – Removing Global Variables (Justification)

Global variables can make programs difficult to manage.
Passing values as parameters improves modularity.
It makes the function reusable with different inputs.
This also improves testing and debugging.
Thus, removing global variables makes the code cleaner.

Task 2 – Refactoring Nested Conditionals (Justification)

Nested if statements make the code hard to read.
Using a simple **if-elif-else structure** improves readability.
It reduces unnecessary nesting.
This makes the logic easier to understand.
Thus, the code becomes simpler and clearer.

Task 3 – Refactoring File Handling Code (Justification)

The same file handling code was repeated multiple times.
This increases code duplication.
Using a function removes repetition.
The **with open()** statement safely handles file closing.
Thus, the code becomes shorter and easier to maintain.

Task 4 – Optimizing Search Logic (Justification)

The original code searches elements using a loop.
This takes more time when the list becomes large.
Using a **set** allows faster searching.
Set lookup is usually faster than list search.
Thus, performance of the program improves.

Task 5 – Refactoring into OOP Design (Justification)

Procedural code places all logic in one place.
Using a class organizes code into objects and methods.
Encapsulation keeps related data together.
This improves readability and reusability.
Thus, OOP design makes the code better structured.

Task 6 – Performance Optimization (Justification)

The original loop checks every number individually.
This takes more time for large ranges.
Using built-in functions like **sum()** is faster.
Python built-in functions are optimized internally.
Thus, the program becomes more efficient.

Task 7 – Removing Hidden Side Effects (Justification)

The original function modifies a global list.
This can cause unexpected changes in the program.
Returning values instead of modifying globals is safer.
It makes the function independent.
Thus, the code becomes more predictable.

Task 8 – Simplifying Input Validation (Justification)

The original code has many nested conditions.
This makes it hard to read and test.
Splitting validation into functions improves clarity.
Each function performs one specific task.
Thus, the code becomes easier to understand and maintain.